🏫 HALF-YEARLY EXAMINATION 2026 🏫

CLASS: IX **SUBJECT:** COMPUTER APPLICATIONS 💻

TIME ALLOWED: 2 HOURS ⏳

MAXIMUM MARKS: 80 💯

GENERAL INSTRUCTIONS TO CANDIDATES (READ CAREFULLY): ⚠️

1. **Reading Time:** You are allowed an additional 15 minutes strictly for reading the question paper. ⏱️ You must **NOT** start writing or marking the paper during this time. The total writing time is exactly 2 hours. 📝
2. **Answer Booklets:** All answers must be written on the separate answer booklet provided to you. 📜 Do **NOT** write any answers, rough work, or pencil marks on this question paper. 🚫
3. **Paper Structure:** This question paper is divided into two strict sections: **Section A** (40 Marks) and **Section B** (40 Marks). 🗂️
4. **Attempt Rules:**
    - **Section A:** Attempt **ALL** questions. This section is compulsory. 🎯
    - **Section B:** Attempt **ANY FOUR** questions out of the given choices. If you attempt more than four, only the first four will be evaluated. The rest will be strictly crossed out without being read. ✂️
5. **Documentation:** For all programming questions in Section B, you must provide a clear variable description table. 📊 Proper indentation in your code is mandatory.
6. **Illegible Handwriting:** If your code, brackets { }, semicolons ;, or operators are illegible or ambiguous, they will be considered incorrect. The examiner will not guess your intentions. ✍️❌

READ THIS DECREE BEFORE ATTEMPTING ANY CODE:

*If **ANY** syntax error is made while writing a program or analyzing a code snippet—whether it is a missing semicolon ;, an unescaped double quote \", a forgotten curly*

*brace }, or a single incorrect uppercase/lowercase letter (Java is strictly case-sensitive)—you will instantly be awarded **0 MARKS** for that entire question.* 💀🪓

*There is absolutely **NO STEP-MARKING** for dead code. If a syntax error exists, the Java compiler crashes fatally. The output is never generated, and the program does not run. A program that does not compile is a completely worthless program.* 📉

*This rule also strictly applies to predicting output (dry runs): If you fail to spot an illegal escape sequence, a type-casting failure, or a fatal compiler crash in a snippet, and you write an output anyway, your dry run is invalid, and your score is **ZERO**.* 💣

*Do not argue with the examiner on Result Day. The marking scheme is absolute. The compiler does not forgive, and neither will We. **NO EXCEPTIONS. NO MERCY.*** 🚒🔪

SECTION A (40 Marks) 📚

QUESTION 1. Name the following: [5 Marks]

a. Interaction of various objects occurs through _____.

b. Applets support graphics through _____ components.

c. .class files in Java are _____.

d. Java package used to localize text elements is _____.

e. Attributes are defined within a _____.

QUESTION 2. [2 Marks]

Write a single Java statement to output the exact text shown below on the screen.

Required Screen Output: "\\n" \t \Java\ is '/fun/' //n \

QUESTION 3. [1 Mark]

Name the phase which allocates memory in the compiler to store different variables.

QUESTION 4. [2 Marks]

Find the final data type result. Also, predict the output of z and showcase your reasoning.

int a = 10; float f = 20f; char c = 'K';

double d = a++ + f-- - c++ + --c;

z = (float)(a/f) - d * c;

QUESTION 5. [2 Marks]

Given the expression i + f - b * c, find the resultant data type (using coercion).

int i; float f; byte b; char c;

Output your answer as flow lines.

QUESTION 6. [6 Marks]

Write the hierarchy and precedence of the following operators:

>>, <<, !=, ^, ~, !

[Note: If any operator is not in sequence, whole marks will be deducted.]

QUESTION 7. [2 Marks]

How can you input a character literal using next().charAt(0)? Explain the reason why this is used and how it works.

QUESTION 8. [8 Marks]

Write a Java program to input one negative decimal number and one integer using the Scanner class. You must maintain:

1. A count variable (to track the no. of operations done) starting at 0.
2. A totalScore accumulator (to store the sum of calculations) starting at 0.0.

Perform the operations in this order only:

- Find the rounded integer value of the negative number using Math.round(). Add the result to totalScore and increase count by 1.
- Find the nearest even integer of the same number using Math.rint(). Add the result to totalScore and increase count by 1.
- Divide the integer input by 2. Add the quotient to totalScore and increase count by 1.
- Find the absolute value of the difference between count and the number 10. Add it to totalScore and increase count by 1.

Display the final totalScore and count with appropriate messages.

QUESTION 9. [4 Marks]

Using the Math.random() function, simulate a dancing engine. There are 3 students: I, II, and III. If the engine outputs 2, Student II should dance for 15-20 seconds. Now, for Student II, the engine must generate how much time he/she should dance (Limit → 1-20 seconds only). Write a program to run this engine.

QUESTION 10. [1 Mark]

Define Source characters.

QUESTION 11. [1 Mark]

State the Unicode value of the '!' character.

QUESTION 12. [4 Marks]

When and where was James Gosling born? When and where did he complete his graduation with a B.Sc. in Computer Science?

QUESTION 13. [2 Marks]

Write a single Java statement to output the exact following text on the screen:
The teacher said, "\t means tab, not /t!"

SECTION B (40 Marks) 💻

QUESTION 14.

(a) [7 Marks]

Write a complete Java program using the Scanner class to calculate and output a corrupted memory dump. The program must strictly perform the following operations in order:

1. **Input:** Accept one negative decimal number named x and one positive integer named y. 📥
2. **Modulo & Casting Trap:** Use a shorthand operator to assign y the remainder of y divided by the sum of: the integer-casted nearest even integer of x (Math.rint()), AND the absolute value of -10. 🧮
3. **Ceiling & Pre-decrement Trap:** Use a shorthand operator to subtract the following expression from x: A **pre-decremented** y minus the mathematical ceiling of x. 📉
4. **Concatenation Output:** Print exactly "V1 = " concatenated with the final values of x and y (Do NOT mathematically add x and y together; glue them as a string). 🖨️
5. **The Escape Sequence Nightmare:** As the absolute final line of your program, you must write a single System.out.println() statement that physically outputs the *exact* text shown below on the screen. (Warning: You must perfectly reverse-engineer the escape sequences required to print these exact slashes and quotes without crashing the compiler!): 🚨

"\n" \t 'C:\new_folder\' \n "/n"

(b) [3 Marks]

Write a complete Java program using the Scanner class to perform the following strictly in order:

1. Accept a single 3-letter uppercase word from the user. 🔠
2. Extract the first character and the last character of the word. ✂️
3. Find the absolute difference between their Unicode values using Math.abs(). 🔢
4. Divide this exact difference by 2 and calculate the mathematical ceiling of the result. Store this in a double variable. 📈
5. Print the final calculated value. 🖨️

QUESTION 15.

(a) [5 Marks]

Write a complete Java program using the Scanner class to simulate the decryption of a digital Quantum Vault. Perform the following operations strictly in order:

1. **The Input:** Accept a double named x (assume the user inputs a negative decimal) and a single char named key. 📥
2. **The ASCII Modulo Trap:** Declare an int initialized as shift = 4. Declare an int named lock. Assign lock the explicitly casted ASCII integer value of key. Immediately use a shorthand operator (%=) to update lock with the remainder of itself divided by a **pre-decremented** shift. 🔠🧮
3. **The Integer Division Minefield:** Use a shorthand operator (+=) to add the division of Math.abs(-9) by 2 to the variable x. *(Lethal Restriction: You are forbidden from typing 2.0 or 9.0. You must write the division using only the integers 9 and 2, but force the compiler to add exactly 4.5 to x instead of chopping it to 4.0!)* ➗💥
4. **The Negative Tie-Breaker:** Reassign x to be the mathematical maximum (Math.max()) between:
    - The nearest even integer of the current x (Math.rint())
    - The integer-chopped value of the current x (Math.round()) 📉⚖️
5. **The Ternary Override:** You are completely forbidden from using if-else. Use a single System.out.println() statement containing a **Nested Ternary Operator (? :)** to print exactly one message: 🚨
    - If lock is exactly 0 **AND** x is strictly less than 0.0, print "Vault Opened." 🔓
    - If lock is NOT 0, but x is equal to -0.0 or 0.0, print "Vault Jammed." 🛑
    - Otherwise, print "Access Denied." ❌

(b) [5 Marks]

Write a Java program to simulate a "Number Casino" using Math.random(). The program must perform the following strictly in order:

1. Generate a random decimal number between **15.0 and 25.0** (inclusive of 15, exclusive of 25) and store it in a variable. 🎲
2. Calculate Score A: Find the **nearest even integer** of this random number. ⚖️
3. Calculate Score B: Find the **Natural Logarithm** of the exact same random number, but strictly **rounded UP** to the nearest integer. 📈
4. Find the **maximum** value between Score A and Score B. 🏆
5. Print the generated random number, Score A, Score B, and the Maximum value with appropriate messages. 🖨️

QUESTION 16.

(a) [5 Marks]

Write a complete Java program using the Scanner class to simulate Virat Kohli's batting performance in a high-pressure match. You must strictly perform the following operations in order:

1. **Input & Initialization:** Accept one **negative decimal number** named swing and one **positive integer** named form. Declare an integer variable named runs initialized exactly to 97. 📥
2. **The Cover Drive (Shorthand & Coercion Trap):** Use a shorthand addition operator (+=) to add the following mathematical expression directly to runs: The nearest even integer of swing (Math.rint()) modulo (%) a **pre-decremented** form. 🧮
3. **The Strike Rate (Math Library):** Create a double variable named strikeRate. Assign it the maximum value (Math.max()) between the absolute value of Math.floor(swing) and the mathematical ceiling of swing. 📈
4. **The Umpire's Decision (Logical & Relational):** Use an if-else statement to print exactly one message based on the following conditions: 🚨
    - If runs is greater than or equal to 100 **AND** strikeRate is strictly greater than 5.0, **OR** if the exact integer rounded value of swing (Math.round()) is exactly equal to -3, print "Century! King Kohli reigns supreme!" 👑
    - Otherwise, print "Out! The crowd goes silent." 😶
5. **Output:** Print the final calculated values of runs, form, and strikeRate. 🖨️

(b) [5 Marks]

💻 Write a complete Java program using the Scanner class. Perform the operations strictly in order:

1. Accept one negative decimal number named alpha and one positive integer named beta. 📥
2. Use a shorthand operator to multiply alpha by the difference between its Math.rint() value and its Math.round() value. ⚡
3. Create a double variable named omega. Assign it the maximum value between the absolute value of Math.floor(alpha) and the square root of beta--. ⛰️
4. Use a shorthand operator to add Math.log(1.0) plus ++beta to the omega variable. ⬆️
5. Use an if-else block with Relational and Logical operators to print exactly one message: 🚨
    - If omega is greater than or equal to 10.0 **OR** beta is not equal to 0, **AND** the mathematical ceiling of alpha is strictly less than 0, print "System Override".
    - Otherwise, print "System Failure".
6. Print the final values of alpha, beta, and omega. 🖨️

QUESTION 17. [10 Marks] 📝

Design a class named CR7Masterclass to simulate a penalty kick against an AI Goalkeeper. The simulation includes dynamic shot physics, randomized Goalkeeper AI, and the Video Assistant Referee (VAR) rebound twist.

Class Name: CR7Masterclass 🏟️

Data Members / Instance Variables: 📦

- double power - Stores the negative kinetic energy of the shot.
- double angle - Stores the exact decimal angle of the strike.
- int attempts - Tracks the number of strikes taken.
- boolean isGoal - Stores true if he scores, false if the Goalkeeper saves it.

Member Methods: ⚙️

1. **void prepareShot():** 📥
    - Use the Scanner class to accept one **negative decimal** for power and one **positive decimal** for angle.
    - Initialize attempts to 0 and isGoal to false.
2. **void executeStrike():** ⚡
    - **The Goalkeeper AI (Random & Casting):** Create an int variable named gkReach. Use Math.random() and explicit type casting to generate a random integer strictly between **5 and 9** (inclusive). 🎲🧤
    - **The Physics Engine (Ternary & Math):** Create a double variable named dip. Use the **Ternary Operator (? :)** to assign its value:
        - If angle is strictly greater than 45.5, assign it the nearest even integer of angle using Math.rint().
        - Otherwise, assign it the mathematical ceiling of power using Math.ceil(). 📈

- **The Shot Logic (Unary & Short-Circuit):** Use a complex logical expression to determine if it is a goal. Assign isGoal = true IF:
    - A **pre-incremented** attempts is less than or equal to 1, **AND** the absolute value of dip is strictly greater than gkReach.
    - **OR**, IF power is NOT equal to the mathematical floor of dip. 🤯

3. **void varRebound():** 📺🔄
    - Check if isGoal is true. If he scored, the method should immediately terminate (do nothing).
    - If isGoal is false, the shot is saved, but VAR spots the keeper off his line! Re-take the penalty:
        - Use a **shorthand operator** to find the remainder (%) of power divided by the maximum value between 3.0 and Math.log(1.0). 🧮
        - Use a **shorthand operator** to subtract a **post-decremented** attempts from the angle. 📉
        - Invoke the executeStrike() method again to take the second shot. ⚽⏱️
4. **void stadiumScreen():** 🖨️
    - Use the **Ternary Operator (? :)** to print exactly one message based on the final isGoal status:
        - If true, print: "SIUUU! The keeper had no chance!" 🐐
        - If false, print: "Saved! Heartbreak in the stadium." 💔
    - Print the final values of power, angle, and attempts.
5. **public static void main(String args[]):** 🏭
    - Create an object of the CR7Masterclass class.

- Call the methods in the strictly correct sequence to accept input, strike the ball, trigger the VAR rebound check, and display the final outcome. 🎯

QUESTION 18 [10 MARKS]

Write a complete Java program using the Scanner class. Perform the following operations strictly in order, without triggering a single syntax error or data-loss warning:

1. **The Input :** Accept a single-word String for the student's name, a double for their rawMarks, and a positive integer for their rollNo. 📥
2. **The ASCII Modulo Strike:** Create an int variable named attitudePenalty. Assign it the explicitly casted ASCII integer value of the **second character** of their name. Immediately use a shorthand operator (%=) to update attitudePenalty with the remainder of itself divided by a **pre-decremented** rollNo. 🔠🧮
3. **The .5 Negative Deathmatch:** Use a shorthand operator (*=) to multiply rawMarks by the mathematical maximum (Math.max()) between the exact nearest even integer of -7.5 (Math.rint()) and the rounded value of -7.5 (Math.round()). 📉⚖️
4. **The Explicit Coercion Cast:** Create an int variable named finalScore. You must explicitly cast the following exact calculation into this integer: Add rawMarks to the mathematical ceiling (Math.ceil()) of the sum of Math.floor(-2.8) and Math.abs(-1.2). *(Warning: Watch your brackets during the cast!)* 🧱💥
5. **The Short-Circuit Pin-Drop (Logical Matrix):** Create a boolean named pinDropSilence. Assign it true IF:
    - attitudePenalty is NOT equal to 0, **AND** a **post-incremented** rollNo is strictly greater than 0.
    - **OR**, if the mathematical minimum of finalScore and 0 is exactly -1.
    - Otherwise, assign it false. 😲
6. **The Teacher's Verdict (Nested Ternary):** You are FORBIDDEN from using if-else here. You must use a **Nested Ternary Operator** (? :) to store a message in a String variable named verdict: 🚨

- If finalScore is greater than or equal to 80, the message is "Perfect. Silence." 🏆
- If it is not, check if finalScore is greater than or equal to 40. If true, the message is "Pass. Next." 📝
- If both are false, the message is "Fail. Leave." ❌

7. **The Output:** Print finalScore, pinDropSilence, and verdict on separate lines. 🖨️